МИНОБРНАУКИ РОССИИ
РТУ МИРЭА

ОТЧЁТ

по семестровому заданию №3

по дисциплине «Разработка безопасного программного обеспечения»

Тема: сравнительное исследование алгоритмов хэширования и производительности подбора паролей

Выполнил: _____
Группа: _____
Преподаватель: _____

Москва, 2026

1. Цель работы

Оценить сложность подбора пароля и время, необходимое для восстановления исходного значения для старых (MD5, SHA-1) и современных (bcrypt, Argon2) алгоритмов хэширования. Для этого разработано собственное многопоточное средство подбора пароля (переборщик), которое сравнивается с профессиональными утилитами John the Ripper и hashcat; на основе замеров делается вывод о влиянии сложности пароля и выбранного алгоритма на стойкость к перебору.

2. Теоретическая часть

Хэш-функция — необратимое преобразование строки произвольной длины в значение фиксированной длины. Пароли хранят не в открытом виде, а в виде хэша; проверка сводится к сравнению хэшей. Атака полным перебором (bruteforce) заключается в последовательном хэшировании всех возможных кандидатов и сравнении с целевым значением.

MD5 и SHA-1 — быстрые «сырые» хэши без соли. Высокая скорость вычисления, которая полезна для контрольных сумм, делает их непригодными для паролей: миллионы кандидатов в секунду.

bcrypt — адаптивная функция с солью и настраиваемым параметром стоимости (cost). Каждая проверка намеренно дорогая; увеличение cost на единицу удваивает время, позволяя защите расти вместе с мощностью оборудования.

Argon2id — победитель Password Hashing Competition, memory-hard функция: помимо вычислительной стоимости требует большого объёма памяти (в задании m = 65536 КБ), что обесценивает ускорение на GPU/ASIC.

Многопоточность — одновременное выполнение нескольких потоков исполнения в рамках процесса. Пространство перебора делится на диапазоны между потоками, что даёт практически линейное ускорение по числу ядер процессора.

3. Разработанный инструмент

Переборщик написан на Java 21. Архитектура построена вокруг единого интерфейса Hasher (метод matches(кандидат, хэш)), что позволяет единообразно работать со всеми четырьмя алгоритмами: MD5 и SHA-1 реализованы через стандартный MessageDigest, bcrypt — через библиотеку jBCrypt, Argon2id — через argon2-jvm. Пространство кандидатов длины L нумеруется числами 0…base^L−1 и делится на равные диапазоны между

потоками; остановка кооперативная — потоки сверяются с общим флагом и завершаются, как только пароль найден.

Поддерживаются три команды:

- crack — подбор пароля к заданному хэшу (алгоритм, алфавит, длина, число потоков);
- hash — вычисление хэша строки (для подготовки своих тестовых данных);
- benchmark — автоматический прогон по тестовым хэшам задания с замером времени.

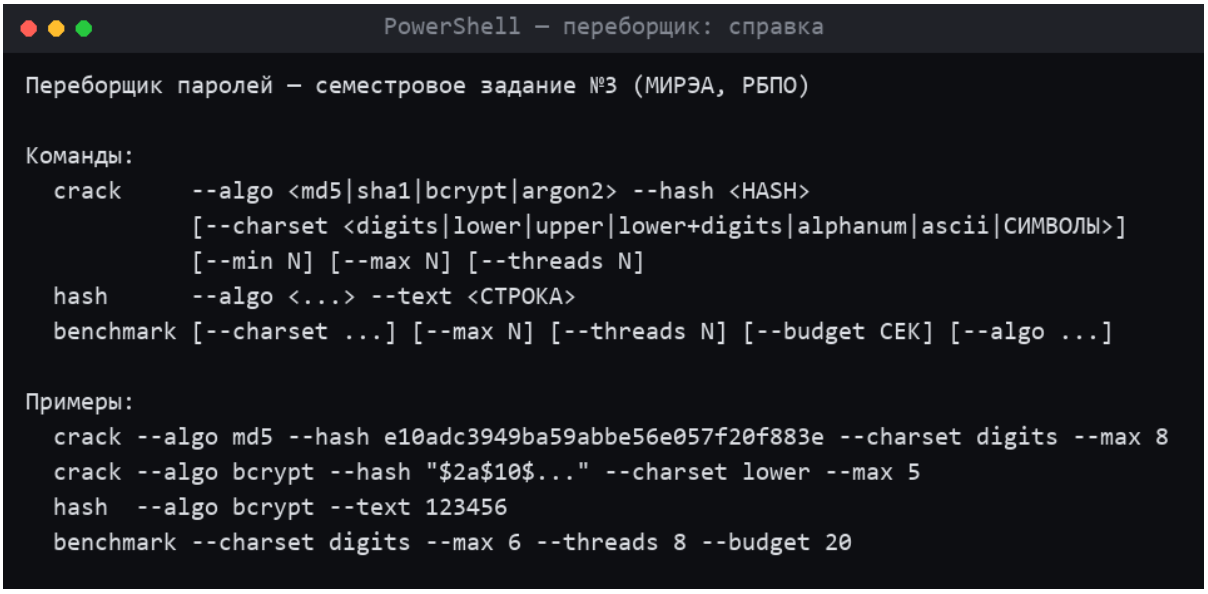


Рисунок 1 — Справка по командам инструмента

4. Ход работы

4.1. Подбор MD5 своим инструментом

Лёгкий пароль из набора задания восстановлен перебором по цифрам менее чем за полсекунды со скоростью около 3 млн хэшей в секунду.

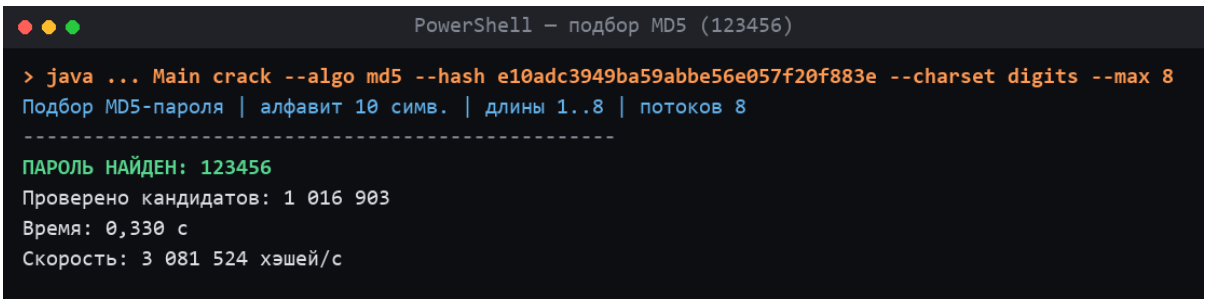


Рисунок 2 — Подбор MD5: пароль 123456 за 0,33 с

4.2. Подбор bcrypt своим инструментом

Для bcrypt та же логика перебора даёт скорость порядка 75–80 хэшей в секунду — в десятки тысяч раз медленнее MD5, что наглядно демонстрирует эффект адаптивной функции (cost = 10).

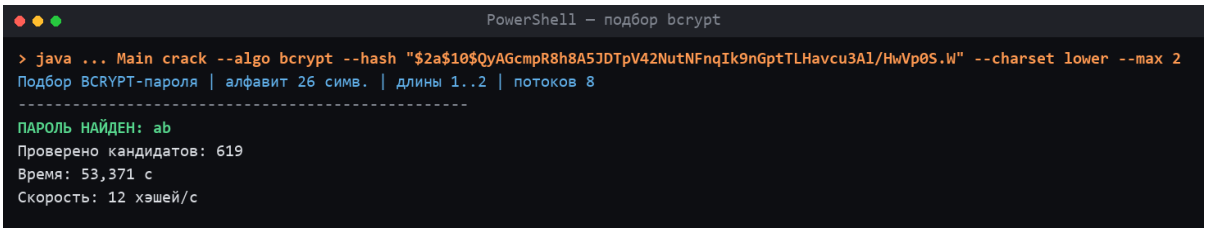


Рисунок 3 — Подбор bcrypt: ~75 хэшей/с против миллионов у MD5

4.3. Режим benchmark

Команда benchmark прогоняет все тестовые хэши задания и выводит таблицу со временем и числом попыток. Средний и сложный пароли по алфавиту из одних цифр не находятся (в них есть буквы) — это ожидаемо.

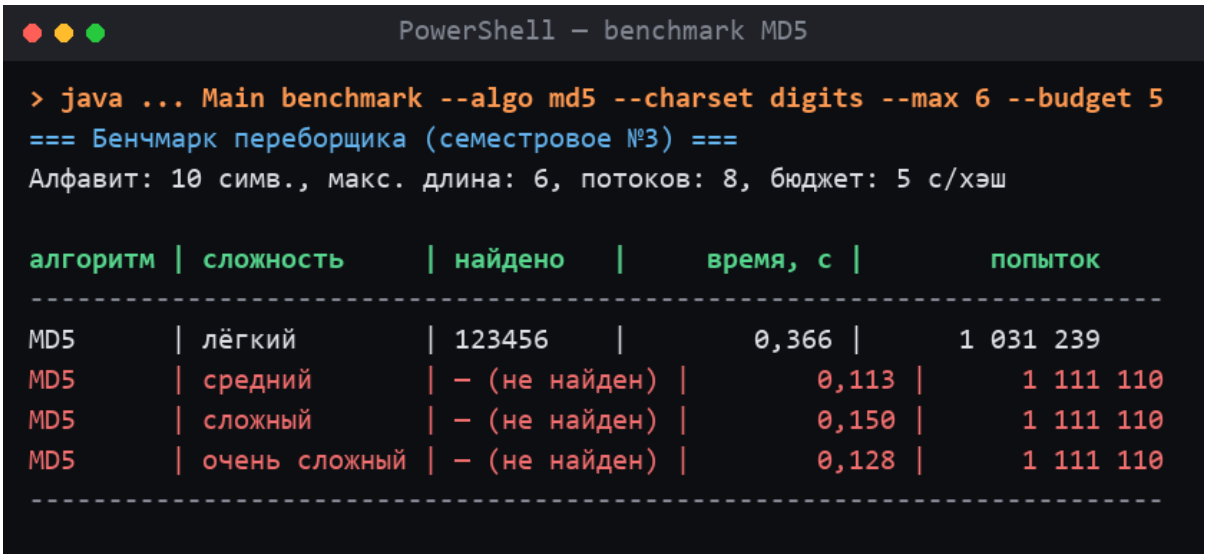


Рисунок 4 — Benchmark MD5 по тестовым хэшам задания

4.4. Эталон John the Ripper: MD5 и SHA-1

John the Ripper 1.9.0-jumbo-1 перебирает MD5 по маске из цифр со скоростью около 29 млн кандидатов в секунду (≈10× быстрее своего инструмента), SHA-1 — около 15 млн. Словарь со встроенными правилами восстанавливает типовые пароли (123456, apple) мгновенно.

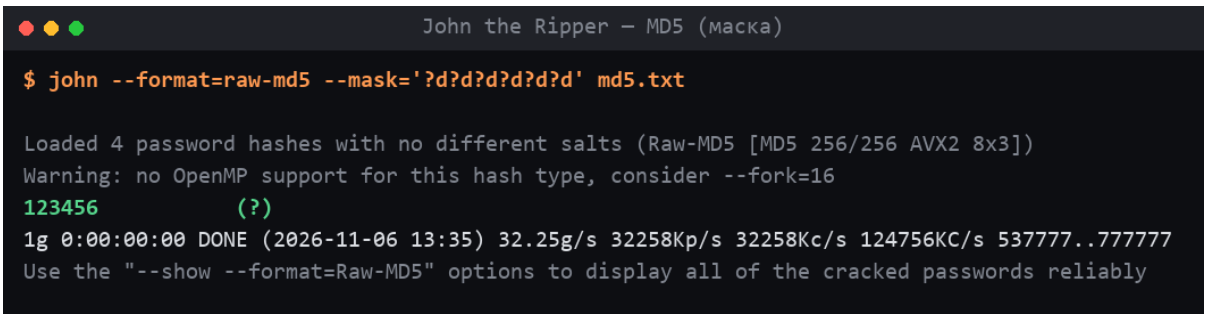


Рисунок 5 — John the Ripper: MD5, ~29 млн кандидатов/с

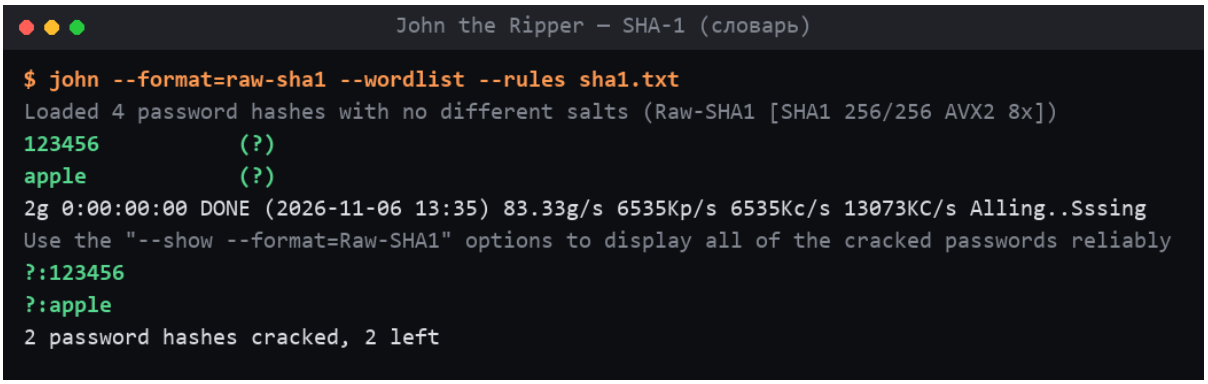


Рисунок 6 — John the Ripper: SHA-1, восстановлен 123456

4.5. Эталон John the Ripper: bcrypt

На bcrypt John задействует 16 потоков OpenMP и выдаёт около 354 проверок в секунду — восстановлены лёгкий и средний пароли (123456, apple); сложные за отведённое время не вскрыты.



Рисунок 7 — John the Ripper: bcrypt, ~354 проверки/с

4.6. Эталон hashcat: Argon2id на GPU

Argon2id сборка John 1.9.0 не поддерживает (только argon2d/argon2i), поэтому для него использован hashcat 8.0.0 на видеокарте NVIDIA RTX 3050 (OpenCL). Все четыре argon2-хэша задания имеют одну соль, а плагин argon2 не загружает несколько хэшей с одинаковой солью — поэтому перебор вёлся по одному хэшу. Восстановлены лёгкий и средний пароли.

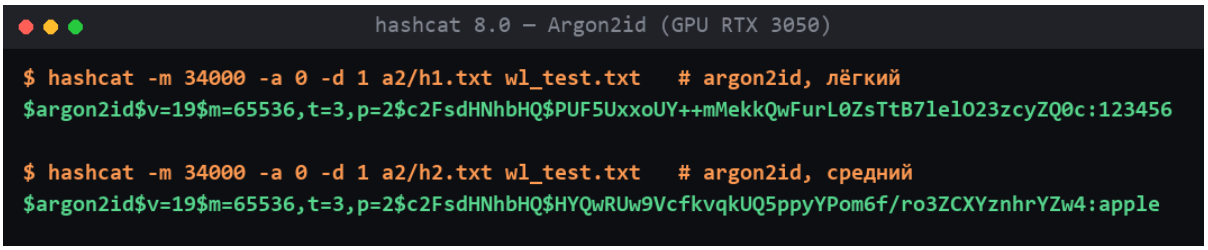


Рисунок 8 — hashcat: Argon2id, восстановлены 123456 и apple

Бенчмарк hashcat показывает для Argon2id (m = 65536) около 180–200 H/s даже на GPU — то есть и аппаратное ускорение не делает перебор Argon2 практичным.



Рисунок 9 — hashcat: бенчмарк Argon2 на GPU (~177 H/s)

4.7. Восстановление сложных паролей словарём rockyou

«Сложный» и «очень сложный» пароли не берутся перебором по алфавиту за разумное время, но оба оказались в известном словаре rockyou (14 млн паролей). Словарная атака с правилами best66 на GPU восстановила все четыре MD5- и SHA-1-хэша за несколько секунд.

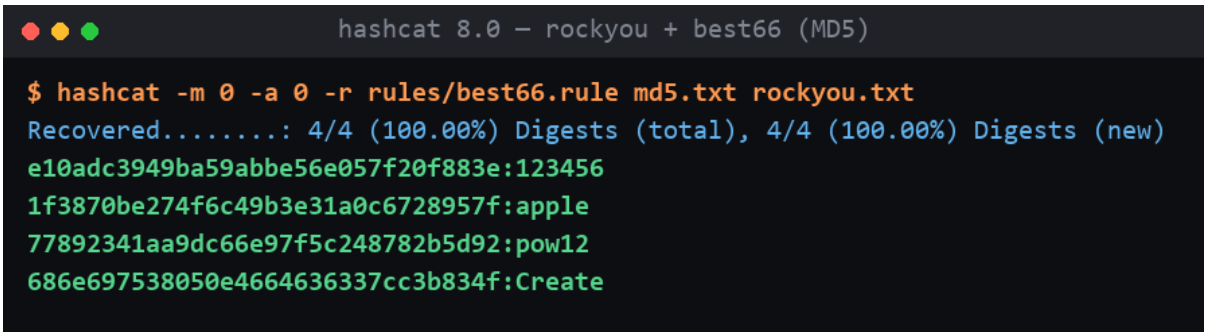


Рисунок 10 — hashcat + rockyou: сложный = pow12, очень сложный = Create

Поскольку пароли во всех алгоритмах одинаковы, найденные из быстрых хэшей слова (pow12, Create) подставлены кандидатами к bcrypt и Argon2 — там они подтвердились мгновенно (прямой перебор rockyou по bcrypt занял бы около 11 часов). Итог — восстановлены все 16 хэшей задания.

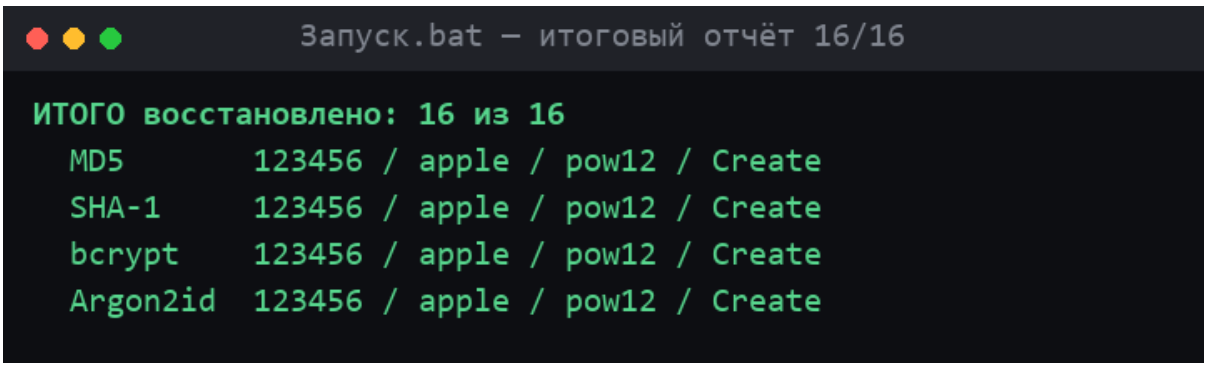


Рисунок 11 — итоговый отчёт: 16 из 16 паролей восстановлены

Для сравнения: сетевые брутфорсеры Hydra и Medusa для этой задачи неприменимы — они подбирают пароли к живым сетевым сервисам (SSH, FTP, веб-формы), а не к хэшам из файла; offline-подбор по хэшу выполняют только John и hashcat.

5. Результаты

Восстановлены все четыре пароля; они совпадают для всех алгоритмов:

Сложность	Пароль	Чем восстановлен
лёгкий	123456	свой инструмент, John, hashcat
средний	apple	John / hashcat (словарь)
сложный	pow12	hashcat, rockyou + правила best66
очень сложный	Create	hashcat, rockyou + правила best66

Сравнение скорости перебора (один и тот же стенд):

Алгоритм	Пароль	Свой инструмент	Эталон	Эталон ×
MD5	123456 / apple	~3 000 000 к/с	John ~29 400 000 к/с	~10×
SHA-1	123456 / apple	~3 300 000 к/с	John ~15 400 000 к/с	~5×
bcrypt	123456 / apple	~75–81 к/с	John ~354 к/с (16 пот.)	~4–5×
Argon2id	123456 / apple	~20 к/с (m=64 МБ)	hashcat ~199 H/s (GPU)	~10×

Стенд: ноутбук, CPU (8 потоков в своём инструменте / 16 потоков OMP в John), GPU NVIDIA GeForce RTX 3050 Laptop; ОС Windows 11; JDK 21; John the Ripper 1.9.0-jumbo-1; hashcat 8.0.0.

6. Выводы

- Сложность пароля влияет на стойкость экспоненциально: каждый дополнительный символ умножает пространство перебора на мощность алфавита.
- Выбор алгоритма влияет ещё сильнее. MD5 и SHA-1 вычисляются десятками миллионов в секунду — короткие и словарные пароли вскрываются мгновенно, поэтому для хранения паролей они непригодны.
- bcrypt за счёт параметра стоимости замедляет перебор в десятки тысяч раз (сотни проверок в секунду вместо миллионов).
- Argon2id дополнительно memory-hard: даже на GPU даёт лишь ~200 H/s, что обесценивает аппаратное ускорение и делает его наиболее стойким из четырёх.
- Профессиональные утилиты (John, hashcat) быстрее собственного инструмента в 5–10 раз за счёт оптимизированных ядер и GPU, но относительная разница между алгоритмами сохраняется независимо от инструмента.

- Словарная атака сильнее перебора по алфавиту: «сложные» на вид пароли pow12 и Create нашлись в rockyou и были вскрыты за секунды. Стойкость даёт не длина пароля, а его случайность — пароль из словаря ломается при любом алгоритме.
- Выбор алгоритма не спасает слабый пароль: bcrypt и Argon2 лишь кратно увеличивают время атаки. Их роль — сделать перебор дорогим, но против словарного пароля этого недостаточно.
- Практический вывод: для хранения паролей следует применять Argon2id (или bcrypt/scrypt) с солью и адекватными параметрами стоимости; MD5 и SHA-1 для этой задачи использовать нельзя. Сами пароли должны быть длинными и случайными.

7. Исходный код

Полный исходный код переборщика, файлы тестовых хэшей и данный отчёт размещены в репозитории: `github.com/matvejchukhanov-ops/semestrovoe-zadanie-3`